

TN SKILLS

Project Documentation Report

A Project Report on

Pocket Smart AI

Your Smart Budget & Recommendation Assistant

Submitted in partial fulfilment of the requirements of the **TN**
Skills Portal Submission

Submitted by

Danasri TS(222509554)
Shalini.M

Academic Year [20 26 - 20 27]

Acknowledgement

We express our sincere gratitude to [Institution / Portal Name] for the opportunity to design, build and document the PocketSmart AI project as part of our academic curriculum.

We would like to thank our faculty mentors and project guides for their valuable feedback, technical guidance and encouragement throughout the design, development and testing of this application.

We also acknowledge the developers of Flask, the Google Generative AI SDK, Chart.js, and the underlying Python ecosystem, whose open-source tools and documentation made the implementation of this project possible.

Finally, we thank our peers and families for their continued support and encouragement during this project.

Abstract

Managing personal finances effectively requires consistent expense tracking, timely budget awareness and actionable, personalized guidance — needs that are often poorly served by spreadsheets or generic banking apps that only present raw transaction data without interpretation. PocketSmart AI addresses this gap by combining a full-featured expense tracker with an AI-driven recommendation engine that converts raw spending data into a financial health score, category-wise insights and specific, actionable savings advice.

The application is built as a full-stack web application using Python and Flask for the backend, HTML, CSS and JavaScript (with Chart.js) for the frontend, and SQLite as the default data store with optional PostgreSQL support for production deployment.

PocketSmart AI supports secure multi-user accounts protected by JWT-based authentication and PBKDF2-HMAC password hashing, allowing each user's expenses, budgets and savings goals to remain fully isolated from other users.

A key design goal of PocketSmart AI is to reduce the manual effort involved in everyday expense logging and financial planning. This is achieved through AI-assisted receipt scanning, which uses the Google Gemini Vision API to automatically extract merchant, amount, category and date from a photographed receipt, and through an AI recommendation engine that analyses spending patterns using Google's Gemini 2.5 Flash model when an API key is configured, gracefully falling back to a built-in local rules engine when it is not.

This report documents the design, architecture, technology stack, implementation, feature set and testing of the PocketSmart AI application, including its authentication system, expense and budget management modules, goal-tracking system, analytics dashboards, and CSV export and reporting capabilities.

The project demonstrates how a modern Generative AI model can be combined with a lightweight, easily deployable Flask web application to deliver a secure, multi-user, AI-powered personal finance assistant that works reliably with or without an active AI subscription.

Table of Contents

Chapter 1: Introduction Chapter 2:
Literature Survey Chapter 3:
System Analysis
Chapter 4: System Architecture and Design Chapter 5:
Technology Stack
Chapter 6: Implementation and Setup
Chapter 7: Feature Description Chapter
8: Testing
Chapter 9: Results, Discussion, Conclusion and Future Enhancements References

List of Figures and Tables

Figure 4.1: High-Level System Architecture of PocketSmart AI
Table 3.1: Software Requirements
Table 3.2: Hardware Requirements Table
4.1: Core Database Tables Table 8.1:
Test Cases and Results

Chapter 1: Introduction

1.1 Overview

Tracking day-to-day spending and understanding where money goes each month is a challenge for most individuals. Traditional approaches — manual notebooks, spreadsheets, or basic banking statements — record transactions but rarely interpret them, leaving the user to manually calculate totals, compare them against a budget, and decide what changes would help.

PocketSmart AI addresses this challenge by combining a structured expense-tracking system with an AI recommendation layer. The application records every transaction against a category and a monthly budget, computes a financial health score, and produces specific, prioritized recommendations — either through Google's Gemini AI models or, when no AI key is configured, through an equivalent built-in rules engine so that the core experience never

depends on an external subscription.

1.2 Problem Statement

Individuals frequently lose track of discretionary spending because expense capture is tedious (manually typing every purchase) and because raw transaction lists do not, by themselves, indicate whether spending is on track against a budget or which category is driving overspending.

Additionally, most existing budgeting tools either require a paid subscription for any AI-assisted insight or provide only static, generic tips rather than recommendations grounded in the user's own category-wise spending, budget utilization and savings trajectory. There is a need for a lightweight, self-hostable application that captures expenses with minimal friction (including receipt-photo capture), enforces per-user data isolation, and produces genuinely personalized guidance regardless of whether a paid AI API key is available.

1.3 Objectives

- To design and implement a Flask-based, multi-user web application that allows individuals to record, edit, search and categorize their personal expenses securely.
- To implement secure account management using JWT-based authentication, HTTP-only session cookies, and PBKDF2-HMAC-SHA256 password hashing.
- To integrate Google's Gemini Vision API for automatic receipt scanning (OCR), extracting merchant, amount, category and date from a photographed receipt.
- To integrate Google's Gemini 2.5 Flash model to analyse spending patterns and generate a financial health score, category insights and personalized recommendations, with an equivalent local rules-engine fallback when no API key is supplied.
- To implement budget management and savings-goal tracking, including monthly limits, category benchmarks, and progress-tracked savings goals with deposit functionality.
- To provide interactive analytics dashboards (7-day spend trend, 6-month trend, category breakdown, budget runway) using Chart.js.
- To provide CSV export of transaction history for offline record-keeping and further analysis.
- To secure the application with rate limiting, standard HTTP security headers, and support for both SQLite (development) and PostgreSQL (production) as the data store.

1.4 Scope of the Project

The scope of this project covers the design and development of PocketSmart AI as a Flask-based web application that allows registered users to log expenses, set a monthly budget, define savings goals, scan receipts using AI-based OCR, and receive AI-generated or rules-based financial recommendations, all presented through interactive charts and a dashboard interface.

The current version does not include bank-account integration, real-time transaction syncing with financial institutions, or native mobile applications; these are identified as potential future enhancements in Chapter 9. The AI-assisted features (OCR and advisor) require a user-supplied Google Gemini API key; without one, the application automatically falls back to its built-in local rules engine so that all core functionality remains available.

1.5 Organization of the Report

This report is organized into nine chapters. Chapter 1 introduces the project. Chapter 2 surveys related work in personal-finance applications and AI-assisted advisory tools.

Chapter 3 presents the system analysis, including feasibility and requirements. Chapter 4 describes the system architecture and design. Chapter 5 details the technology stack.

Chapter 6 covers implementation and setup. Chapter 7 describes the application's features in detail. Chapter 8 presents the testing approach and results. Chapter 9 discusses results, advantages, limitations, conclusions and future enhancements, followed by references.

Chapter 2: Literature Survey

Prior to designing PocketSmart AI, a review of existing personal-finance tracking applications, AI-assisted advisory tools, and OCR-based expense-capture systems was conducted to identify design opportunities and gaps that this project could address.

2.1 Personal Finance and Budgeting Applications

Widely used budgeting applications typically allow manual or bank-linked transaction entry, category tagging, and simple budget-versus-actual comparisons presented through charts. These tools are effective at recording data but generally stop short of interpreting it — the user must still decide that unaided, which category to cut back on or whether their current trajectory is sustainable for the month.

2.2 AI-Powered Financial Advisory Tools

A newer generation of finance apps has begun layering large language model (LLM) based chat assistants on top of transaction data to answer natural-language questions such as 'where did I overspend this month?'. These integrations are frequently offered only as a premium, subscription-gated feature, and the advice generated is often generic rather than computed from the user's specific budget utilization, category benchmarks and savings rate.

2.3 Receipt Scanning and OCR-Based Expense Capture

Optical character recognition has long been used to digitize printed receipts, traditionally through dedicated OCR engines followed by rule-based field extraction. Multimodal generative AI models such as Google's Gemini Vision now allow a single API call to both read the receipt image and return structured fields (merchant, amount, category, date) directly, substantially simplifying the OCR pipeline compared with legacy text-extraction-plus-parsing approaches.

2.4 Identified Gaps

The survey identified the following gaps: (1) a lack of lightweight, self-hostable finance trackers that combine secure multi-user accounts with AI-based insight generation without mandating a paid third-party subscription; (2) limited use of multimodal AI for direct receipt-to-transaction capture in open, easily deployable applications; and (3) AI recommendations in most tools are generic rather than computed against the user's own category benchmarks, budget remaining, and savings rate.

PocketSmart AI addresses these gaps by pairing a locally-computed financial health score and rules-based recommendation engine with an optional, user-supplied Gemini API key that upgrades both the recommendations and the receipt-capture experience — ensuring the application is fully functional, secure and genuinely personalized with or without a paid AI subscription.

Chapter 3: System Analysis

3.1 Proposed System

The proposed system, PocketSmart AI, is a Flask-based, multi-user web application that presents a single-page dashboard where a signed-in user can add, edit, search, filter and delete expenses, set a monthly budget, track savings goals, and view AI-generated or rules-based financial recommendations alongside interactive charts.

Each user's data (expenses, budget, goals) is isolated at the database level and is only accessible after successful authentication. The generated recommendations, health score and advisor responses are displayed within the web interface, and the user can continue the session by asking follow-up questions to the AI advisor, adjusting the monthly budget, or exporting a CSV of their transactions.

3.2 Feasibility Study

3.2.1 Technical Feasibility

The technology stack used in PocketSmart AI — Python, Flask, HTML/CSS/JavaScript with Chart.js, SQLite/PostgreSQL, and the Google Gemini SDK — is well-documented, widely adopted and does not require specialized hardware. The project is technically feasible for a small team to design, implement and deploy within an academic project timeline.

3.2.2 Economic Feasibility

PocketSmart AI relies primarily on open-source software (Python, Flask, SQLite) and, optionally, the Google Gemini API, which offers a free usage tier sufficient for personal and demonstration use. No paid infrastructure is strictly required to run or evaluate the application, making it economically feasible for individual users and academic deployment alike.

3.2.3 Operational Feasibility

The application's dashboard-based interface requires no technical knowledge from the end user — a user simply registers an account, logs expenses (manually or via receipt scan), and reviews the automatically generated insights and charts. The optional 'Preset Demo Data' feature allows a new user to explore all features instantly using realistic sample transactions, goals and a budget.

3.3 Software and Hardware Requirements

Table 3.1: Software Requirement s

Requirement	Detail
Operating System	Windows 10/11, macOS, or Linux
Runtime	Python 3.9 or later
Package Manager	pip
Backend Framework	Flask 3.0
Front-End	HTML5, CSS3, JavaScript, Chart.js
AI SDK	Google Generative AI SDK (google-genai)
Database	SQLite (built-in) or PostgreSQL (via psycopg2-binary)
Development Tools	Visual Studio Code or any Python IDE
Version Control	Git and GitHub

Table 3.2: Hardware Requirements

Requirement	Detail
Processor	Intel Core i3 / AMD equivalent or higher (minimum)
RAM	4 GB minimum, 8 GB recommended
Storage	Less than 500 MB for dependencies and application files
Internet Connectivity	Required only for Gemini AI features (OCR and AI advisor)

3.4 Development Methodology

The project followed a Rapid Application Development (RAD) approach, well suited to a small, student-built application constructed and refined over successive iterations. Early iterations focused on core expense CRUD operations and budget tracking; later iterations added authentication, receipt OCR, the AI recommendation engine, advanced charts, CSV

export and security hardening (rate limiting, security headers, PostgreSQL support).

Each iteration involved refining the rules engine and AI prompts to ensure that recommendations remained specific, quantified (expressed in currency and percentages) and actionable, rather than generic financial advice.

Chapter 4: System Architecture and Design

4.1 Architecture Overview

PocketSmart AI follows a layered client-server architecture centered around a Flask backend application that mediates between the browser-based frontend, the relational database, and the Google Gemini API. All authenticated requests pass through a JWT verification layer before reaching the expense, budget, goal, or AI-recommendation endpoints.

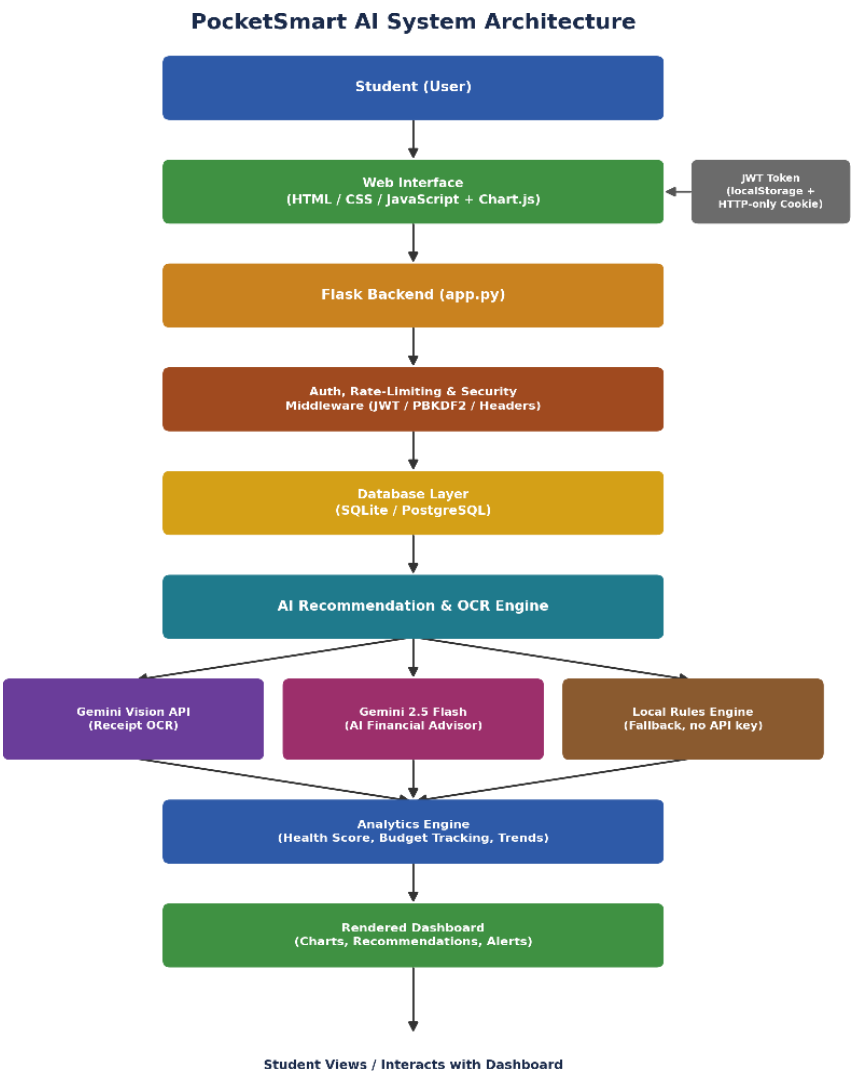


Figure 4.1: High-Level System Architecture of PocketSmart AI

4.2 Component-wise Architecture Description

4.2.1 Presentation Layer (HTML / CSS / JavaScript)

The presentation layer is implemented using templates/index.html, styled with static/ style.css (supporting both light and dark themes), and made interactive via static/app.js, which handles authentication forms, the expense modal, charts (rendered with Chart.js), the AI advisor drawer, and receipt-scan capture.

4.2.2 Authentication and Security Layer

This layer issues and verifies JSON Web Tokens (7-day validity), stores tokens in an HTTP-only cookie in addition to the Authorization header, hashes passwords with PBKDF2-HMAC-SHA256 (260,000 iterations, random salt per user), applies a sliding-window in-memory rate limiter to sensitive endpoints, and attaches standard security headers (X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, Referrer-Policy, Permissions-Policy) to every response.

4.2.3 Backend / API Layer (Flask - app.py)

The backend, implemented in app.py, exposes RESTful routes for authentication, expense CRUD, budget configuration, goal management, receipt OCR, AI recommendations/advisor chat, and CSV export. Each route validates its inputs, enforces per-user data scoping via the authenticated user's ID, and returns structured JSON responses.

4.2.4 Database Abstraction Layer

A thin abstraction layer normalizes SQL parameter placeholders and cursor handling so the same application code can run against SQLite (the default, file-based store) or PostgreSQL (selected automatically when a DATABASE_URL environment variable is present), allowing the same codebase to serve both local development and production deployment.

4.2.5 AI Recommendation and OCR Engine

When a Gemini API key is supplied, this layer sends a structured financial snapshot (totals, category breakdown, six-month trend, recent transactions) to the Gemini 2.5 Flash model and parses its JSON response into a headline, health score, recommendations and savings tips; for receipt scanning, it sends the receipt image to Gemini Vision and parses the returned merchant, amount, category and date. When no API key is supplied, or if the AI call fails, the application transparently falls back to an equivalent local rules engine that computes the same fields from the user's own spending data.

4.2.6 Analytics and Health-Score Engine

This component computes the financial health score (starting at 100 and deducted for budget overrun and per-category overspending against fixed benchmarks), the seven-day and six-month spending trends, the projected month-end total, the savings rate, and the category-wise breakdown consumed by both the dashboard charts and the

recommendation engine.

4.3 Data Flow

The data flow within PocketSmart AI proceeds as follows: (1) the user registers or logs in and receives a signed JWT; (2) the user adds an expense manually or by scanning a receipt, which the frontend sends to the appropriate API endpoint; (3) the backend validates and persists the record, scoped to the authenticated user; (4) the summary and analytics engine recomputes totals, category breakdowns and the health score; (5) the AI recommendation engine (Gemini or local rules) generates insights from this summary; and (6) the frontend renders the updated KPIs, charts and recommendations without a full page reload.

Because each request is stateless and authenticated independently via the JWT, the system scales horizontally across multiple worker processes without requiring server-side session storage.

4.4 Use Case Description

The primary actor in the PocketSmart AI system is the registered User. The key use cases supported include Register / Login, Add / Edit / Delete Expense, Scan Receipt (AI OCR), Set Monthly Budget, Create / Deposit into Savings Goal, View Dashboard and Analytics, Request AI Recommendations, Ask the AI Advisor a Question, and Export Expenses as CSV.

For example, in the 'Scan Receipt' use case, the user uploads or captures a receipt image; the application sends the image to Gemini Vision, receives structured fields, and pre-fills the add-expense form for the user to review and confirm before saving.

4.5 Database Design

Table 4.1: Core Database Tables

Table	Purpose
users	Stores username, email, and salted PBKDF2 password hash for each registered account.
expenses	Stores each transaction's title, amount, category, note, date and owning user_id.
budget	Stores the current monthly spending limit per user.
goals	Stores savings goals per user, including the target amount and amount saved so far.

Chapter 5: Technology Stack

PocketSmart AI is intentionally built using a minimal but production-capable technology stack, prioritizing security, rapid development and straightforward deployment.

5.1 Frontend / Interface

- HTML5: Structures the single-page dashboard, authentication screens, expense

modal, and AI advisor drawer.

- CSS3: Implements a complete design system with light and dark themes, KPI cards, charts panels and responsive layout.
- JavaScript: Handles authentication flow, asynchronous API calls, chart rendering, receipt capture, and dynamic DOM updates.
- Chart.js: Renders the seven-day trend bar chart, category doughnut chart, and six-month trend line chart.

5.2 Core Application (Backend)

- Python: The core programming language used to implement the entire backend application logic.
- Flask 3.0: A lightweight Python web framework used to implement REST API routes, authentication middleware, and request handling.
- SQLite3 / psycopg2: Database drivers used for the default SQLite store and the optional PostgreSQL production store respectively.

5.3 Artificial Intelligence Stack

- Google Gemini API: The underlying multimodal large language model used for receipt OCR (Gemini Vision) and financial advisory recommendations (Gemini 2.5 Flash).
- Google Generative AI SDK (google-gen ai): The official Python SDK used to authenticate with and send requests to the Gemini model family.
- Local Rules Engine: A deterministic, Python-based fallback that reproduces the same recommendation structure without requiring an external API call or key.

5.4 Data Persistence

PocketSmart AI uses SQLite (spend_guide.db) as its zero-configuration default data store, ideal for local development and demonstration. When a DATABASE_URL environment variable is supplied, the application automatically switches to PostgreSQL, which is better suited for concurrent, multi-user production deployments.

5.5 Security

- JWT (HMAC-SHA256): Stateless authentication tokens with a 7-day expiry, verified on every protected request.
- PBKDF2-HMAC-SHA256 (260,000 iterations): Salted password hashing to protect stored credentials.
- Sliding-window rate limiter: Restricts request volume per IP address on

authentication, OCR, and AI-advisor endpoints.

- HTTP security headers: X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, Referrer-Policy, and Permissions-Policy applied to every response.

5.6 Development Tools

- requirements.txt: Declares and pins Python package dependencies (Flask, google-genai, and the optional psycpg2-binary).
- Git and GitHub: Used for version control and source code management throughout development.
- Visual Studio Code: The primary development environment for app.py, the HTML templates, and static assets.

Chapter 6: Implementation and Setup

This chapter describes the practical setup and implementation procedure for running PocketSmart AI, based on the project's structure of app.py, templates/index.html, static/ app.js, static/style.css and requirements.txt.

6.1 Environment Configuration

PocketSmart AI can run with zero configuration using its SQLite default. Optional environment variables allow customization: DATABASE_URL switches the application to PostgreSQL; GEMINI_API_KEY pre-configures the AI features so users are not required to enter a key in the interface; and SECRET_KEY sets a fixed JWT signing secret, which is strongly recommended for any persistent or production deployment so that issued tokens remain valid across server restarts.

6.2 Installation and Launch Steps

Step 1 — Clone or download the repository — the developer obtains the PocketSmart AI project files, including app.py, requirements.txt, templates/ and static/.

Step 2 — Create a virtual environment (recommended) — a dedicated Python virtual environment is created for the project to isolate its dependencies.

Step 3 — Activate the virtual environment — the environment is activated (for example, using "venv\Scripts\activate" on Windows or "source venv/bin/activate" on macOS/Linux).

Step 4 — Install dependencies — running "pip install -r requirements.txt" installs Flask and the Google Generative AI SDK; "pip install psycpg2-binary" is additionally required only when using PostgreSQL.

Step 5 — (Optional) Configure environment variables — the developer may export DATABASE_URL, GEMINI_API_KEY and SECRET_KEY as described in Section 6.1.

Step 6 — Launch the application — running "python app.py" initializes the database schema and starts the Flask development server, accessible at <http://localhost:5000>.

Step 7 — Register an account — the user creates an account through the sign-up form, or signs in if an account already exists.

Step 8 — Explore the dashboard — the user can click "Preset Demo Data" to instantly load realistic sample transactions, goals and a budget, or begin adding real expenses, scanning receipts, and requesting AI recommendations immediately.

6.3 Application Internals

Internally, app.py is organized around a small number of core responsibilities: initializing the database schema on startup, issuing and verifying JWTs, exposing authentication routes, exposing CRUD routes for expenses, budget and goals, computing the analytics summary (health score, trends, category breakdown), calling the Gemini API or local rules engine for recommendations, handling receipt OCR, and exporting CSV reports.

This modular, single-file backend design keeps the application easy to understand, debug and extend, which is well suited to an academic project of this scope while still demonstrating production-relevant patterns such as authentication, rate limiting and database abstraction.

Chapter 7: Feature Description

This chapter describes the core features of PocketSmart AI in detail, building on the architectural description provided in Chapter 4.

7.1 Multi-User Authentication

Users can register with a username, email and password, and sign in with either identifier. Passwords are never stored in plain text; they are salted and hashed using PBKDF2-HMAC-SHA256. On successful login, a signed JWT is issued and stored both client-side and as an HTTP-only cookie, ensuring each user only ever sees their own expenses, budget and goals.

7.2 Expense Tracking

Users can add, edit and delete expenses, each recorded with a title, amount, category (Food, Transport, Shopping, Bills, Entertainment, Health, or Other), an optional note, and a date. The transaction table supports free-text search, category filtering, month filtering, and sorting by date or amount.

7.3 Receipt Scanning via AI OCR

Users can photograph or upload a receipt, which is sent to the Google Gemini Vision API to automatically extract the merchant's name, total amount, likely category, a brief note, and the transaction date, pre-filling the add-expense form for quick confirmation. If no Gemini

API key is configured, the system returns a manual-entry stub, so the workflow remains usable without AI.

7.4 Budget Management

Users can set and update a monthly spending limit. The dashboard displays the remaining budget, a progress bar of spend against the limit, and a projected month-end total based on the user's current daily spending rate.

7.5 Savings Goals Tracker

Users can create named savings goals with a target amount, track progress as funds already saved, and record deposits toward each goal over time, supporting longer-term financial planning alongside day-to-day budgeting.

7.6 AI-Powered Recommendations and Financial Health Score

PocketSmart AI computes a 0–100 financial health score based on budget utilization and category-wise overspending, and generates prioritized recommendations (alerts, savings opportunities, and habit suggestions) with quantified potential impact, either via the Gemini 2.5 Flash model or the equivalent local rules engine.

7.7 Conversational AI Advisor

Users can ask free-text financial questions (for example, about food spending, savings goals, or budget limits); the advisor engine detects the intent behind the question and returns a direct, data-grounded answer computed from the user's own transactions and budget.

7.8 Dashboards and Visual Analytics

The dashboard presents KPI cards (total spend, monthly spend, savings rate, daily average), a seven-day spending bar chart, a category-wise doughnut chart, a six-month trend line chart, and per-category budget-runway bars, all rendered with Chart.js and updated live as data changes.

7.9 CSV Export and Reporting

Users can export their full transaction history, or a specific month's transactions, as a downloadable CSV file for offline record-keeping, tax preparation, or further analysis in spreadsheet tools.

7.10 Security and Rate Limiting

All protected endpoints require a valid JWT. A sliding-window in-memory rate limiter caps request volume per IP address on sensitive endpoints such as login, registration, OCR and AI-advisor calls, and every response carries standard HTTP security headers to reduce common web-application attack surface.

Chapter 8: Testing

Testing for PocketSmart AI focused on validating both the functional correctness of the Flask application and the reliability of its AI-assisted and fallback features.

8.1 Levels of Testing

8.1.1 Functional Testing

Functional testing verified that registration, login, and CRUD operations on expenses, budgets and goals behave correctly, including input validation for required fields, positive amounts, and valid categories.

8.1.2 Security Testing

Security testing confirmed that protected endpoints reject requests without a valid JWT, that passwords are never returned in API responses, that rate limits are enforced after the configured threshold is exceeded, and that a user cannot access or modify another user's expenses, budget or goals.

8.1.3 AI and Fallback Testing

Testing confirmed that when a valid Gemini API key is supplied, AI recommendations and OCR return correctly structured JSON, and that when no key is supplied (or the Gemini call fails), the application transparently and correctly falls back to the local rules engine without breaking the user experience.

8.1.4 Usability and Error-Handling Testing

Testing verified that the dashboard renders correctly with both empty and populated data (via the Preset Demo Data feature), and that invalid input (for example, a non-numeric amount or an unrecognized category) produces a clear, user-facing error message rather than a server crash.

8.2 Test Cases

Table 8.1: Test Cases and Results

Test Case ID	Description	Expected Result	Status
TC-01	Register a new account with valid details	Account created; JWT issued and dashboard loads	Pass
TC-02	Log in with an incorrect password	Authentication rejected with a clear error message	Pass
TC-03	Add an expense with a negative amount	Validation error shown; no record created	Pass
TC-04	Access /Api/expenses without a token	401 Unauthorized returned	Pass
TC-05	Request AI recommendations with	Local rules engine returns a	Pass

	no Gemini key set	complete recommendation set	
TC-06	Scan a receipt image with a valid Gemini key	Merchant, amount, category and date auto-extracted	Pass
TC-07	Exceed the login rate limit within one minute	429 Too Many Requests returned after the threshold	Pass
TC-08	Deposit funds into an existing savings goal	Goal's saved amount updates and persists correctly	Pass
TC-09	Export current month's expenses as CSV	CSV file downloads with correctly filtered transactions	Pass
TC-10	Set a monthly budget lower than current spend	Dashboard reflects negative remaining budget and low-runway alert	Pass

Chapter 9: Results, Discussion, Conclusion and Future Enhancements

9.1 Results and Discussion

PocketSmart AI was successfully implemented and demonstrated to reliably capture user expenses (manually and via AI-based receipt OCR), compute an accurate financial health score and budget runway, and generate specific, quantified recommendations through either the Gemini AI engine or the local rules-engine fallback.

The dual-engine design — Gemini AI when a key is available, and an equivalent local rules engine otherwise — proved valuable in ensuring the application remains fully functional and genuinely useful without depending on a paid third-party subscription, directly addressing the gap identified in the literature survey.

The separation of concerns across authentication, expense management, analytics and AI recommendation layers also made it straightforward to add features such as PostgreSQL support and CSV export without disrupting the core application logic.

Overall, the results confirm that a lightweight, Flask-based architecture combined with the Google Gemini API is well suited to building a secure, multi-user, AI-powered personal finance assistant that is inexpensive to run and simple to deploy.

9.2 Advantages

- Fully functional core experience even without a Gemini API key, thanks to the local rules-engine fallback.
- Secure multi-user design with JWT authentication, salted password hashing, and per-user data isolation.
- AI-assisted receipt scanning significantly reduces the manual effort of expense logging.
- Quantified, actionable recommendations (with a currency-based potential

impact) rather than generic financial tips.

- Supports both SQLite (zero-configuration) and PostgreSQL (production-grade), easing the path from prototype to deployment.
- Low operating cost, relying on the free tier of the Gemini API and free or low-cost Flask hosting options.

9.3 Limitations

- The application does not currently integrate directly with bank accounts or payment providers; all expenses must be entered manually or via receipt photo.
- As with any LLM-based system, occasional inconsistencies in AI-generated recommendations or OCR extraction can occur, particularly for unusual or low-quality receipt images.
- The in-memory rate limiter resets on server restart and does not synchronize across multiple worker processes without an external store such as Redis.
- The current version does not send proactive notifications (for example, budget-threshold alerts) outside of the dashboard itself.

9.4 Conclusion

This project presented the design, architecture, and implementation of PocketSmart AI, an AI-powered personal finance assistant that combines secure multi-user expense tracking with AI-assisted receipt scanning and personalized, data-grounded financial recommendations.

The application successfully fulfils its stated objectives: enabling secure, multi-user expense and budget management, reducing manual data entry through AI-based OCR, and generating specific, actionable financial guidance through either the Gemini AI engine or an equivalent local rules engine.

PocketSmart AI serves as a practical demonstration of applying modern Generative AI to everyday personal-finance management, and provides a solid foundation for further enhancement, including bank integration, mobile applications, and richer, longer-horizon financial planning tools.

9.5 Future Enhancements

- Bank and payment-provider integration for automatic transaction import, reducing manual entry to near zero.
- Native mobile applications (iOS/Android) with push notifications for budget-threshold and goal-progress alerts.
- Redis-backed distributed rate limiting and caching for multi-worker, high-traffic production deployments.

- Multi-currency support for users tracking expenses across more than one currency.
- Recurring-expense detection and automatic subscription tracking.
- Exportable PDF financial reports in addition to the existing CSV export.
- Shared or family budgets allowing multiple users to track a joint monthly budget and goals.

References

[1] Flask Official Documentation — <https://flask.palletsprojects.com>

[2] Google Gemini API Documentation — <https://ai.google.dev>

[3] Google Generative AI SDK Documentation — <https://ai.google.dev/gemini-api/docs/sdks>

[4] Chart.js Documentation — <https://www.chartjs.org/docs/latest/>

[5] PostgreSQL Official Documentation — <https://www.postgresql.org/docs/>

[6] Python Official Documentation — <https://docs.python.org/3/>

[7] PocketSmart AI Project Repository — [insert repository / submission link]